Teoría de Examen y Preguntas Cortas (Versión Imprimible)

Conceptos Clave de Calificación, Arquitectura Técnica y Simulacro de Examen Resuelto

Módulo Profesional:	Desarrollo de Interfaces (0488)	**Ciclo Formativo:**	DAM 2 (Grado Superior)
Alumno / Autor:	Daniel Daroca	**Repositorio / Despliegue:**	github.com/danidaroca/DI-Actividad-1 | di-actividad-1.vercel.app

TEMA 1: Arquitectura Angular Standalone e Ionic Framework

• **Definición de Standalone Component:** Componente independiente que no pertenece a ningún NgModule. Se declara con standalone: true en su decorador @Component e importa explícitamente sus propias dependencias en la propiedad imports: [...].
• **Ventajas Técnicas Fundamentales:**
1. **Eliminación de Boilerplate:** Se prescinde de AppModule y módulos intermediarios complejos.
2. **Tree-Shaking Optimizado:** El compilador detecta y elimina código no invocado de manera mucho más granular.
3. **Carga Perezosa Directa (Lazy Loading):** Permite enrutar hacia el componente directamente con loadComponent: () => import(...).
• **Punto de Entrada:** El arranque se ejecuta en main.ts mediante bootstrapApplication(AppComponent, { providers: [...] }).

TEMA 2: Sistema de Enrutamiento y Functional Guards

A partir de Angular 15+, los guards basados en clases con implements CanActivate se consideran obsoletos. Se emplean **Functional Guards** tipados (funciones puras) que resuelven servicios con inject().

DIFERENCIA TÉCNICA CLAVE DE EXAMEN: CanMatchFn vs CanActivateFn

• **CanMatchFn (Fase de Emparejamiento):** Se evalúa antes de que el enrutador intente resolver el componente perezoso. Si devuelve false, **el navegador NUNCA descarga el bundle JavaScript por la red**. Esto logra **Zero-Leakage** (evita que usuarios no autorizados descarguen código confidencial) y ahorra datos móviles.
• **CanActivateFn (Fase de Activación):** Se ejecuta después de que el bundle ya ha sido descargado en memoria. Comprueba si el componente se puede instanciar en el DOM; se utiliza para validar sesión y ejecutar la **redirección contextual** guardando la URL intentada (queryParams: { returnUrl: state.url }).

• **Ciclo de Vida de Navegación:** canMatch (descarta ruta y bloquea descarga de chunk) → Descarga perezosa (loadComponent) → canActivate (valida permisos de sesión en memoria) → Inicialización del Componente → canDeactivate (previene abandono con datos sin guardar).

TEMA 3: HTTP Interceptors Funcionales (HttpInterceptorFn)

• **Concepto y Registro Global:** Middleware basado en el patrón *Chain of Responsibility* configurado en main.ts mediante provideHttpClient(withInterceptors([authInterceptor, errorInterceptor])).
• **Inmutabilidad Obligatoria:** Las instancias de HttpRequest son inmutables por diseño para garantizar determinismo en reintentos y cachés. Para inyectar cabeceras es imprescindible clonar la petición: const clon = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });.
• **Manejo Centralizado de Errores de Seguridad (RxJS catchError):**
- **HTTP 401 (Unauthorized):** Token inválido o expirado. Limpia localStorage y redirige al usuario a /login?returnUrl=... (o dispara el flujo de *Silent Refresh Token* si el backend lo soporta).
- **HTTP 403 (Forbidden):** Sesión válida pero carece de permisos de rol (violación de RBAC). Redirige a ruta segura (/inventory) mostrando aviso sin destruir la sesión.

TEMA 4: Control de Acceso Basado en Roles (RBAC) y Seguridad

• **Autenticación (AuthN) frente a Autorización (AuthZ):**
- *Autenticación:* Responde a '¿Quién eres?'. Verifica la identidad declarada mediante credenciales (login) o la firma digital y expiración de un JWT.
- *Autorización:* Responde a '¿Qué tienes permitido hacer?'. Comprueba los privilegios o permisos asignados al rol en cada vista o endpoint.
• **Modelo RBAC Formal:** Mapea $Usuario \to Rol (ADMIN, SUPERVISOR, USER) \to Permisos Granulares$. Desacopla permisos de los usuarios concretos.
• **Claims del Token JWT:** Datos estructurados en el payload (sub: ID de usuario, email, role, permissions, exp, iat). Se decodifican en AuthService para exponer Signals reactivas.

Enfoque de Seguridad	Descripción Técnica	Evaluación / Decisión Arquitectónica
Flag Booleano Simple (isAdmin: boolean)	Flag binario en el objeto usuario.	**Frágil y no escalable:** Colapsa al añadir roles intermedios (ej. Supervisor, Auditor) y obliga a alterar el modelo de datos.
ABAC (Attribute-Based) (Control por Atributos)	Evalúa reglas dinámicas por contexto en runtime (IP, horario, departamento).	**Sobrecarga excesiva:** Complejidad computacional desproporcionada para clientes móviles y PWAs.
RBAC Formal con Signals (Solución Implementada)	Roles asociados a matrices de permisos estructurados con Angular Signals.	**Óptimo para producción:** Escalable, claro y resolución instantánea $O(1)$ en consultas locales de interfaz.

• **Control Visual Condicional (UI/UX Preventivo):** Ocultar opciones en lugar de bloquear tras el clic:
- Menú lateral: @if (user.role === 'ADMIN') para ocultar la opción del Cuadro de Mando.
- Tabla de datos: [canDelete]='isAdmin' para no dibujar el botón rojo de borrado a usuarios estándar.

TEMA 5: Formularios Reactivos y Validación Asíncrona

• **Validador Asíncrono (AsyncValidatorFn):** Función que retorna Observable<ValidationErrors | null> para comprobar datos contra el backend con latencia simulada.
• **Los 3 Estados del Control (AbstractControl.status):**
1. **PENDING:** Comprobando en el servidor. La UI bloquea el botón y muestra <ion-spinner>.
2. **INVALID:** Duplicado (ej. { emailTaken: true }). La UI muestra alerta roja y deshabilita el botón.
3. **VALID:** Dato disponible (null). La UI muestra check verde y activa el botón de envío.

TEMA 6: Programación Reactiva con RxJS y Angular Signals

• **BehaviorSubject:** Subject con valor inicial que emite de inmediato su último valor a cualquier suscriptor. Se usa para la sesión de usuario y flujo de datos reactivo.
• **forkJoin (Orquestación Concurrente):** Recibe múltiples Observables (métricas, catálogo SQL y notificaciones NoSQL), los ejecuta en paralelo y espera a que **todos completen** para emitir un único resultado agregado (equivalente a Promise.all). Permite usar un único spinner de carga para todo el Dashboard.
• **Angular Signals:** Primitivo reactivo que almacena un valor y notifica directamente a la vista sin sobrecarga de Zone.js (signal(), computed()). Complejidad algorítmica $O(1)$.
• **Componentes Desacoplados y ChangeDetectionStrategy.OnPush:** Componentes reutilizables que solo reciben @Input() y emiten @Output(). OnPush indica a Angular que solo compruebe cambios si cambian las referencias de los Inputs, ahorrando ciclos de CPU.

TEMA 7: Visualización con Chart.js y Prevención de Fugas de Memoria

• **Integración:** Se asocia a un elemento <canvas #chartCanvas> en el DOM mediante @ViewChild y se instancia en el hook ngAfterViewInit().
• **PREGUNTA CRÍTICA: Fugas de Memoria (Memory Leaks):** Chart.js mantiene referencias internas al contexto 2D del Canvas y suscriptores a eventos de ventana (resize, tooltips, animaciones). **Es obligatorio invocar explícitamente chartInstance.destroy() en el hook ngOnDestroy()**. De lo contrario, cada navegación acumula instancias en RAM hasta colapsar el navegador.

TEMA 8: Exportación de Informes, Git Flow y Despliegue SPA

• **Exportación a PDF:** Generación en cliente con jsPDF y jspdf-autotable en formato horizontal (*Landscape*) con membrete corporativo, bloques de KPIs y tabla con bandas alternas.
• **Exportación a CSV con UTF-8 BOM:** Se antepone la marca **BOM (\uFEFF)** al crear el Blob de datos para que **Microsoft Excel** reconozca tildes, caracteres especiales y el símbolo del euro (€) sin corromper el texto.
• **Metodología Git Flow (Opción --no-ff):** *No Fast-Forward* fuerza a Git a crear siempre un commit de fusión explícito, preservando la trazabilidad del árbol histórico de ramas (main, develop, feature/*).
• **Despliegue SPA en Vercel (vercel.json):** En una SPA, el enrutador de Angular opera en el navegador. La regla rewrites: [{"source": "/(.*)", "destination": "/index.html"}] fuerza al servidor web a entregar siempre index.html para que al recargar en rutas profundas (ej. /dashboard) no se produzca un error HTTP 404.

PARTE 2: SIMULACRO DE EXAMEN (10 PREGUNTAS CORTAS RESUELTAS)

Respuestas modelo oficiales con los términos técnicos y justificaciones que exigen los correctores de DAM:

■ PREGUNTA 1: ¿Cuál es la diferencia fundamental entre CanMatchFn y CanActivateFn en Angular?

Respuesta Modelo: La diferencia radica en la fase de enrutamiento y en el impacto de red sobre el *Lazy Loading*. **CanMatchFn** se evalúa antes de cargar el componente perezoso (loadComponent); si devuelve false, **el navegador jamás descarga el chunk JS por la red**, garantizando *Zero-Leakage* y ahorro de ancho de banda móvil. En cambio, **CanActivateFn** se ejecuta después de haber descargado el chunk en memoria para comprobar si se inicializa en el DOM, usándose para validación de sesión y redirección contextual con returnUrl.

■ PREGUNTA 2: ¿Por qué las peticiones HTTP en un HttpInterceptorFn deben clonarse y no modificarse directamente?

Respuesta Modelo: Porque las instancias de HttpRequest en Angular son **inmutables por diseño** para asegurar determinismo frente a reintentos concurrentes y respuestas cacheadas. Para propagar cabeceras de autorización (Authorization: Bearer <token>) es obligatorio invocar el método de clonación inmutable **req.clone({ setHeaders: { ... } })**.

■ PREGUNTA 3: ¿Qué estrategia de manejo centralizado de errores debe aplicar un interceptor ante códigos HTTP 401 y 403?

Respuesta Modelo: Mediante catchError: ante un **HTTP 401 (Unauthorized)** asume sesión expirada, revoca el token local de localStorage y redirige al usuario a /login guardando la ruta actual como returnUrl (o dispara un flujo de *Silent Refresh*). Ante un **HTTP 403 (Forbidden)** reconoce identidad válida pero sin permisos de rol suficientes (violación de RBAC); redirige a una ruta operativa segura (ej. /inventory) mostrando una alerta sin cerrar la sesión.

■ PREGUNTA 4: En el modelo de seguridad RBAC, ¿cuál es la diferencia técnica entre Autenticación y Autorización?

Respuesta Modelo: Autenticación (AuthN): Responde a *'¿Quién eres?'*. Comprueba criptográficamente la identidad declarada mediante credenciales (login) o la firma y vigencia de un token JWT. **Autorización (AuthZ):** Responde a *'¿Qué tienes permitido hacer?'*. Evalúa los privilegios o permisos asignados al rol del usuario autenticado (extraídos de los *claims* del token) para autorizar accesos a vistas (ej. /dashboard) o acciones operativas (ej. borrar registros).

■ PREGUNTA 5: ¿Cuáles son los tres estados de un formulario reactivo durante la validación asíncrona y qué debe mostrar la UI?

Respuesta Modelo: 1. **PENDING:** Petición en curso contra el servidor. La UI bloquea el botón y muestra un indicador de espera (<ion-spinner>).
2. **INVALID:** Validación fallida (ej. { emailTaken: true }). La UI despliega alerta roja de error y mantiene deshabilitado el envío.
3. **VALID:** Validación superada con null. La UI muestra confirmación visual (check verde) y habilita el botón de envío.

■ PREGUNTA 6: ¿Qué problema crítico de rendimiento provoca no destruir una instancia de Chart.js y cómo se resuelve?

Respuesta Modelo: Provoca una **fuga de memoria (*Memory Leak*)**, ya que Chart.js retiene referencias al contexto 2D del Canvas y suscriptores a eventos globales de la ventana (resize, animaciones y puntero). Debe resolverse obligatoriamente en el hook de ciclo de vida **ngOnDestroy()** invocando el método **chartInstance.destroy()**.

■ PREGUNTA 7: ¿Para qué se utiliza el operador forkJoin de RxJS en el servicio de datos del Dashboard?

Respuesta Modelo: Para **orquestar peticiones asíncronas concurrentes** en paralelo (métricas globales, catálogo SQL y notificaciones NoSQL). forkJoin espera a que **todos los Observables completen** y emite una única respuesta agregada, permitiendo mostrar un único spinner de carga y renderizar el Dashboard en un solo ciclo atómico.

■ PREGUNTA 8: ¿Por qué es necesario anteponer la marca BOM (\uFEFF) al generar un archivo CSV para su descarga?

Respuesta Modelo: El **Byte Order Mark (BOM: \uFEFF)** indica explícitamente a aplicaciones cliente como **Microsoft Excel** que el contenido entrante está codificado en UTF-8. Sin esta marca inicial, Excel asume por defecto codificación ANSI/Windows-1252, corrompiendo caracteres con tildes, la letra 'ñ' y el símbolo del euro (€).

■ PREGUNTA 9: ¿Qué función cumple la opción --no-ff al fusionar ramas en la metodología Git Flow?

Respuesta Modelo: La opción --no-ff (*No Fast-Forward*) fuerza a Git a **crear siempre un commit de fusión dedicado (*merge commit*)**, evitando que avance el puntero de forma lineal. Esto preserva de forma explícita la trazabilidad histórica de cuándo una rama de característica (feature/*) se integró en develop o main.

■ PREGUNTA 10: ¿Por qué es imprescindible configurar reglas de reescritura (rewrites) en vercel.json para desplegar una SPA?

Respuesta Modelo: Porque en una **Single Page Application (SPA)** no existen archivos físicos HTML para cada ruta en el servidor. La regla de reescritura redirige internamente cualquier petición que no sea un archivo estático hacia **/index.html**. De este modo, si el usuario recarga la página en rutas profundas como /dashboard, el servidor entrega index.html en lugar de devolver error HTTP 404, permitiendo que el enrutador de Angular tome el control.

■ GLOSARIO RÁPIDO DE PALABRAS CLAVE QUE SUMAN PUNTOS EN EL EXAMEN

- **Zero-Leakage:** Con CanMatchFn no se descarga el bundle por red si no tiene permisos.
- **Inmutabilidad:** Con req.clone({ setHeaders: ... }) en interceptores HTTP.
- **Memory Leak:** Fuga de memoria si no se llama a chart.destroy() en ngOnDestroy().
- **BehaviorSubject:** Mantiene el último valor y lo emite inmediatamente a nuevos suscriptores.
- **forkJoin:** Espera a que todos los Observables en paralelo completen (equivalente a Promise.all).
- **BOM UTF-8 (\uFEFF):** Evita caracteres corruptos al abrir CSV en Microsoft Excel.
- **No Fast-Forward (--no-ff):** Crea merge commits explícitos para trazabilidad en Git Flow.
- **SPA Rewrites:** Redirige a index.html en el hosting para evitar error 404 al recargar.